



THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

Project Proposal

BharatOS: A Unified Developer API Platform for Indian Civic Data

Course Name & Code: UCS503 – Software Engineering

Degree & Year: B.E. (3rd Year)

Institute: Thapar Institute of Engineering and Technology

Submitted By:

Shivansh Singh	– 1024240135
Palveen Kaur	– 1024240155
Rajuss Sachdev	– 1024240106
Shivam Chauhan	– 1024240006

Academic Year: 2026–27 ODD SEMESTER

1 Higher-order goal

... secondary framing

This project aims to make India’s public civic and administrative data easier for software developers to discover, integrate, and reuse, without depending on paid cloud services, ad-hoc scraping, or one-off manual downloads. The broader motivation is digital public infrastructure: reducing the friction between the country’s growing body of open government data and the applications that could be built on top of it. The immediate engineering objective is narrower and fully within course scope: build **BharatOS**, a unified REST API layer that ingests a small set of permitted public/open civic datasets, normalizes them into a single consistent schema, and serves them to developers through authenticated, versioned, rate-limited endpoints with source attribution and freshness metadata — using only data sources we are authorized to use, and without bypassing or impersonating any official government system.

2 Time-to-value

... primary heuristic

- The platform is built in independently testable layers: data ingestion, normalization/storage, and API serving. Each layer produces a verifiable artifact (a populated table, a passing validation check, a working endpoint) before the next one begins.
- We prioritize fast feedback over completeness: the authentication-and-gateway path is proven first with a single dummy endpoint returning static JSON behind a validated API key, before any real dataset is ingested, so integration issues in the request path surface early.
- Success in the first sprint means one canonical endpoint (e.g. PIN code lookup) returning a correctly normalized, source-attributed response end to end. From there, we grow the dataset and endpoint coverage sprint by sprint.

3 Problem Statement

India already publishes a large amount of public data across government departments, ministries, state portals, the Open Government Data (OGD) platform, and API Setu. However, data *availability* does not automatically mean developer *accessibility*. A developer who wants to build even a simple civic application typically has to work across sources that differ in:

- **Structure:** different schemas, field names, and file formats (CSV, JSON, ad-hoc APIs) for conceptually the same data.
- **Operations:** different update schedules, missing metadata, and inconsistent geographic naming (e.g. “Bangalore” vs. “Bengaluru” vs. “BENGALURU”).
- **Access:** different authentication mechanisms, rate limits, and API conventions, with no single, documented developer experience.

As a result, developers can spend more time on data discovery and cleaning than on the application itself.

Problem: How can Indian public civic data be transformed into a standardized, developer-friendly API platform that applications can consume through a single interface?

Why this matters now (contextual relevance): As more civic-tech, research, and public-service applications look to build on open government data, a thin, well-documented abstraction layer over that data can noticeably lower the barrier to building new, legitimate applications on top of it.

4 Proposed Solution

... *Software Proposition*

4.1 Overview

BharatOS is a **developer-oriented API platform** with three decoupled layers:

- **Developer platform:** registration, login, API-key generation/revocation, a usage dashboard, and OpenAPI documentation.
- **Civic data API:** standardized, versioned REST endpoints (e.g. `/v1/locations/pincode/{pincode}`, `/v1/locations/districts`) returning consistent JSON with source and freshness metadata attached to every response.
- **Data pipeline:** ingestion, validation, cleaning, normalization, deduplication, and meta-data extraction from permitted public/open sources into a shared schema.

A thin platform-infrastructure layer, authentication, rate limiting, caching, logging, versioning, and monitoring, sits underneath all three and is what turns a data pipeline into a usable product.

4.2 Core workflow

1. **Ingestion** (offline, scheduled): a connector reads a permitted dataset or public API in its native format.
2. **Normalization** (offline): the record is validated, cleaned, mapped onto BharatOS's canonical schema, deduplicated, and tagged with source and freshness metadata, then stored in PostgreSQL.
3. **Serving** (live): a developer's request passes through the API gateway, API-key validation, a rate-limit check, and a cache check; on a cache miss the data service queries PostgreSQL, the response is cached in Redis, and a standardized JSON envelope is returned.
4. **Feedback:** the request is logged, and the developer can see it reflected in their usage dashboard.

4.3 Operational constraints for an educational, from-scratch setting

- BharatOS will not bypass government authentication, access private/restricted government databases, or scrape restricted portals; it will not claim to be an official Government of India service, and it will not modify source data.
- It will not provide identity verification (Aadhaar, PAN, driving licence, etc.); restricted or currently inaccessible sources are represented through clearly labelled mock/sandbox adapters for academic demonstration only.
- The system will store only the personal information needed to operate developer accounts, and will never present stale cached data as if it were current.
- Datasets stay out of Git version control, since they do not fit GitHub's practical size limits; code lives in the repository, the data separately, with its source and expected local layout documented so the pipeline is reproducible.

5 Solution Approach

... *Engineering focus*

This is an engineering course project, so the focus is on a correct, reproducible ingestion-to-serving pipeline and a clean API contract, rather than inventing new datasets or novel algorithms.

5.1 Data sources

... non-research

Initial coverage draws only from India's existing open-data ecosystem, principally the Open Government Data (OGD) platform and API Setu, which already expose a range of civic datasets and APIs. BharatOS does not aim to replace these; it sits above them as a normalization and developer-experience layer, starting with a small, well-scoped set of categories, such as PIN codes, states, and districts, before adding health, education, transport, environment, or agriculture data.

5.2 Data ingestion and normalization pipeline

Each external source gets its own adapter (OGD adapter, generic API adapter, file adapter) behind a common *Data Source Interface*, so a new source can be added without rewriting the platform. Every record passes through the same pipeline: collect → parse → validate → normalize → deduplicate → enrich metadata → store → index → serve. Normalization is where naming inconsistencies (e.g. the Bangalore/Bengaluru example above) are mapped to a single canonical value while the original source value is preserved.

5.3 API gateway and backend

... web-based solution

A Node.js/TypeScript backend implements the API gateway: every request is authenticated against a hashed API key, checked against a Redis-backed rate limit and cache, and, on a cache miss, served from PostgreSQL. Endpoints are explicitly versioned (`/v1/...`, later `/v2/...`) so schema changes do not silently break existing integrations, and every response uses one of two standard envelopes: a `data/meta` success envelope carrying source and freshness information, or a standard `error` envelope with a machine-readable code.

5.4 Frontend and developer portal

A Next.js/TypeScript developer portal handles registration, login, API-key creation and revocation, OpenAPI/Swagger documentation with try-it-out examples, and a usage dashboard (requests today, success rate, latency, per-endpoint traffic). A separate admin view gives visibility into users, dataset status, error rate, and data freshness. The portal talks to the backend only through the same public API contract, so frontend and backend can be built and tested independently.

5.5 Testing and fast-delivery enabler

Each layer is independently buildable and testable: unit tests cover API-key validation, normalization logic, and rate-limit calculations; integration tests exercise the gateway together with authentication, PostgreSQL, and Redis; security tests cover invalid/expired keys, unauthorized endpoints, and malformed or excessive requests; and performance tests measure latency and throughput at increasing concurrency (10, 50, 100, and 500 concurrent requests). Together these catch pipeline breakage early rather than at demonstration time.

6 Evaluation Criterion

... measurable and attributable

Success is defined by measurable metrics tied to the core request path, not by aspirational feature counts.

6.1 Primary evaluation metric

API correctness and responsiveness under load: the fraction of a defined functional test suite (covering registration, authentication, key revocation, rate limiting, and standard data/error responses) that passes, combined with response latency (p_{95}) measured under increasing concurrent load.

- **Target:** 100% of the defined functional test suite passing, with p_{95} latency and error rate reported from actual measured runs at each load level rather than assumed in advance.
- **Attribution:** measured directly by the automated test suite and the performance-test harness (10/50/100/500 concurrent requests), logged per run.

6.2 Secondary evaluation metrics

- **Cache effectiveness:** Redis cache hit ratio for repeated requests to the same endpoint/parameters.
- **Data provenance coverage:** percentage of API responses that carry complete source and freshness metadata.
- **Developer time-to-first-call:** time for a new developer to register, generate an API key, and receive a successful response, used as a proxy for documentation quality.
- **Reliability:** the system returns an explicit, standardized error rather than silently serving incorrect or stale data when an upstream source is unavailable.

6.3 Pilot validation plan

... *high-level*

- Recruit a small group of developers unfamiliar with the project and time them through registration, key generation, reading the docs, and making a first successful call.
- Run the full functional test suite (registration, auth, revoked/invalid keys, rate limiting, valid/invalid endpoints, cache hit/miss, malformed requests) and the performance-test harness at each pilot milestone, and record actual pass/fail and latency numbers rather than projected ones.

7 Scalability

... *with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** new data sources are added through the adapter interface without touching the API layer, and the gateway is stateless per request, with rate-limit counters and cache state held in Redis, so it can grow from a handful of endpoints to many civic-data categories, and from a single REST API toward richer interfaces (GraphQL, geospatial queries) without a redesign.
2. **Operationally deployable (practically):** the platform runs on Docker containers and commodity cloud infrastructure, on top of PostgreSQL and Redis, so it stays reproducible for grading and for every teammate, with a straightforward path from a single pilot instance to multiple horizontally scaled API instances behind the same gateway.

8 Engine availability heuristic

The tooling needed already exists and is freely available, so effort goes into the platform, not infrastructure:

- Next.js and Node.js/TypeScript for the developer portal, dashboards, and API gateway/backend.
- PostgreSQL (relational + JSONB) and Redis for storage, caching, and rate-limit counters.
- REST, OpenAPI, and Swagger UI for a documented, versioned API surface.
- Docker and standard cloud infrastructure for reproducible deployment.
- India's existing Open Government Data platform and API Setu as the underlying, already-public data ecosystem BharatOS normalizes and re-exposes, rather than duplicates.

Using these mature, freely available components lets effort go into the ingestion pipeline, normalization logic, and API contract, rather than rebuilding basic infrastructure that already exists.

9 Project scope and deliverables

... iteration-friendly

9.1 Initial deliverable

... first iteration

- Developer registration/login and API-key generation and revocation (minimal dashboard).
- An ingestion and normalization pipeline for at least one canonical dataset (e.g. PIN code / state / district lookup), validated and stored in PostgreSQL.
- Versioned REST endpoint(s) under `/v1/locations/...` returning the standardized JSON envelope with source and freshness metadata.
- API-key authentication and Redis-backed rate limiting, with standardized error responses.
- A minimal OpenAPI specification / documentation page.

9.2 Subsequent deliverables

- Additional civic-data categories (health, education, transport, environment, agriculture) behind the same adapter interface.
- A full developer usage-analytics dashboard and an admin monitoring dashboard (error rate, data freshness, source availability).
- An automated test suite covering unit, integration, security, and performance testing, run against defined test cases with a recorded pass/fail report.
- Data-quality scoring per dataset (freshness, completeness, consistency), and, as stretch goals, an SDK (@bharatos/sdk), a GraphQL surface, and multilingual dataset metadata.

10 Risks and mitigations

- **Unauthorized API access / credential theft:** mitigated with API-key authentication, hashed key storage, and HTTPS/TLS in transit.
- **API abuse or traffic spikes:** mitigated with Redis-backed rate limiting and per-tier quotas, returning a standard 429 response with retry guidance.

- **Data corruption from heterogeneous sources:** mitigated by validating and normalizing every record, with database constraints and parameterized queries, before it is written or served.
- **Upstream source failure or staleness:** mitigated with monitoring and a graceful fallback to cached data, always surfaced through explicit freshness/status metadata rather than presented as current.
- **Legal/policy exposure from ambiguous or restricted sources:** mitigated by scoping data strictly to permitted public/open datasets and officially exposed APIs, and representing any restricted service through a clearly labelled mock/sandbox adapter for academic purposes only.

11 Summary

BharatOS proposes a developer-first civic-data API platform: one normalization and API layer over a small set of permitted Indian public/open datasets, rather than another one-off citizen-facing application. It meets the course criteria through fast, independently verifiable pipeline stages (ingestion, normalization, serving), a request path with automated unit, integration, security, and performance tests, and a measurable evaluation criterion centered on functional correctness and response latency under load. It stays within a disciplined, from-scratch scope, our own ingestion pipeline, our own normalization schema, our own API gateway, while leaving a clear, adapter-based path toward more data categories, an SDK, and richer query interfaces as future work.